

Design Note — RAG Voice Assistant for Political Campaign AI

1. Problem & Approach

Political campaigns span large geographies — each district has its own schemes, candidates, and FAQs. A voter in Vijayawada needs different answers than one in Rayalaseema. Static IVR trees can't handle this at scale.

We ruled out fine-tuning immediately — it requires days of compute, breaks on document updates, and still hallucinates. RAG separates knowledge from reasoning. The LLM stays frozen; domain knowledge lives in a vector store, updated by uploading documents. Every answer is grounded in retrieved source chunks and traceable back to a file. For voice, speech is transcribed to text first and then treated identically to a typed query.

2. Architecture

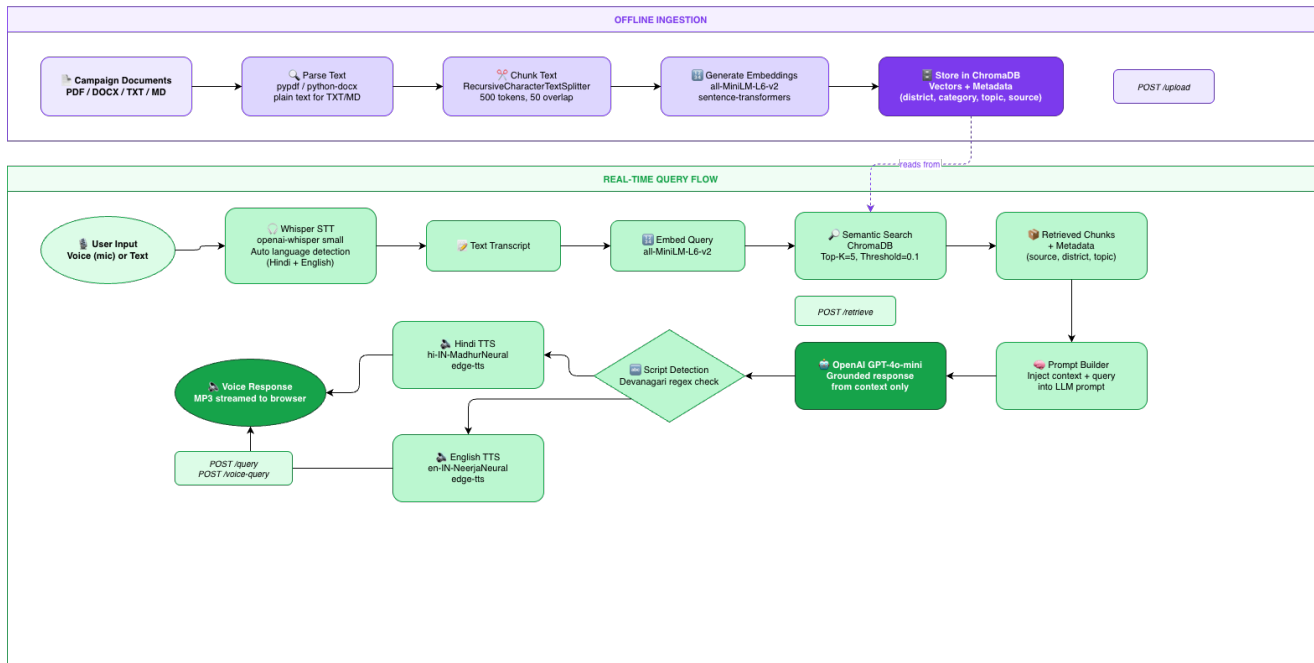
Two pipelines that never overlap — ingestion (offline, slow, once per document) and query (real-time, fast, every request). Keeping them separate ensures uploads never block queries.

Ingestion: Upload → Parse (pypdf / python-docx) → Chunk (500t, 50 overlap) → Embed (all-MiniLM-L6-v2) → Store (ChromaDB)

Query: Voice/Text → Whisper STT → Embed query → ChromaDB similarity search → Top-K chunks → GPT-4o-mini → Script detect → edge-tts → Audio response

The architecture is intentionally modular — every component (vector DB, LLM, STT, TTS) can be swapped without touching the rest.

RAG Voice Assistant — System Architecture



3. Key Engineering Decisions

Stage	Chosen	Alternatives Considered	Reason
Chunking	RecursiveCharacterTextSplitter 500t/50	Fixed-size, semantic chunker	Respects paragraph structure; fixed-size cuts mid-sentence
Embeddings	all-MiniLM-L6-v2	OpenAI text-embedding-3-small,	Free, local, no API cost, sufficient for short campaign text
Vector DB	ChromaDB local	Pinecone, Qdrant, FAISS	Zero infrastructure — runs on any
LLM	gpt-4o-mini	GPT-4o, Claude Sonnet, Llama 3	Fast and cost-effective; full GPT-4o is overkill when context is already
STT	Whisper small	Whisper base, Deepgram, AssemblyAI	small outperforms base on Hindi and Hinglish; streaming
TTS	edge-tts neural voices	gTTS, ElevenLabs, Piper	Free Microsoft neural voices, no API key, far more natural than gTTS
Language	Devanagari regex	langdetect library	Deterministic — regex on Unicode range never misclassifies short

4. Challenges

- **Silent ingestion failure** — uploads returned HTTP 200 but stored nothing in ChromaDB due to a missing `return len(chunks)`. Caught only when retrieval returned empty results. Fix: always verify store contents via `/debug` before debugging the retrieval layer.
- **Similarity threshold tuning** — small sample docs produced scores compressed between 0.3–0.5 regardless of relevance. Threshold lowered to 0.1 for prototype. In production with a real corpus, scores spread naturally and 0.3–0.5 is appropriate.
- **Hinglish TTS** — mixed Hindi-English responses caused mispronunciation on both voices. Production fix requires a normalization layer splitting response by script before TTS. Defaulted to English TTS for mixed-script output given timeline.
- **Environment conflict** — `uvicorn` picked up system Python 3.13 instead of conda env Python 3.11. Fixed with `python -m uvicorn`.
- **FastAPI mount conflict** — `app.mount()` intercepted the root `/` route. Fixed by registering the `/` handler before the mount call.

5. Latency & Streaming Strategy

Current (prototype):

- Full audio file uploaded after user stops recording
- Whisper transcribes the complete file
- Retrieval starts only after full transcript is ready
- LLM waits for full retrieval before generating
- TTS waits for full LLM response before speaking

Production approach:

- Replace Whisper with a streaming STT service (Deepgram / AssemblyAI) that emits partial transcripts via WebSocket as the user speaks

- Retrieval triggers on the first 3-4 words — no need to wait for the full sentence
- 300ms debounce applied to avoid redundant searches on every single word
- Retrieval results update as more words arrive and the query becomes more specific
- By the time the user finishes speaking, retrieval is already warmed up with the right chunks
- LLM call starts immediately with pre-retrieved context — no cold retrieval delay
- LLM streams tokens back rather than waiting for full completion
- TTS speaks each sentence as it arrives rather than waiting for the full answer
- Net latency reduction: ~800ms off end-to-end response time

6. Scalability

Current setup is single-process, local, no auth. Production changes: ChromaDB → Pinecone or Qdrant (concurrent writes, replication), local embeddings → hosted endpoint (unblocks event loop), Whisper → streaming STT, document ingestion → background job queue (Celery + Redis), local disk → S3, add API key auth on all endpoints. The architecture doesn't need to change — just swap components.